

Parallelization of Full-Field Reorientation in Microstructure Evolution

Chun-Mei Yang
314551115

Jyun Lai
314553031

Chi-Yi Lin
314581003

Abstract

Full-field microstructure evolution simulations, such as the Cellular Automaton (CA) based OMicroN framework, are computationally intensive due to the repetitive evaluation of grain boundary migration rates. This work addresses the performance bottleneck by parallelizing the core reorientation kernel using a hybrid approach of OpenMP for multi-core CPUs and CUDA for GPUs. We identify that the original `std::set` data structure inhibits parallel efficiency and implement a “Set-to-Vector” flattening strategy to enable random-access iteration and improve memory locality.

Extensive scalability tests were conducted on grid sizes ranging from 50×50 to 500×500 . The OpenMP implementation achieves a **3.54x speedup on 4 threads** and scales to **over 5x on 8 threads**, effectively mitigating the computational load up to the memory bandwidth limit. Furthermore, our experimental CUDA implementation exploits the massive parallelism of the NVIDIA T4 GPU, achieving a remarkable **246x reduction in per-step kernel execution time** compared to the serial baseline. Our analysis reveals a distinct transition from compute-bound to memory-bound behavior at large grid sizes ($> 300 \times 300$), demonstrating that heterogeneous computing is essential for overcoming the “Memory Wall” in large-scale materials simulations.

1 Introduction

Recrystallization and grain growth phenomena are fundamental mechanisms determining the mechanical properties, texture evolution, and microstructural stability of metallic alloys, particularly interstitial-free (IF) steels. During annealing, the migration of grain boundaries—driven by the reduction of stored energy and capillary forces—defines the final microstructure. Accurately simulating these kinetics is critical for optimizing thermomechanical processing routes in industrial applications.

Full-field microstructure models based on Cellular Automaton (CA) methods have emerged as a powerful tool to study these topological evolutions. Unlike mean-field or Monte Carlo approaches [3, 5], CA models preserve explicit spatial connectivity, enabling the study of local curvature effects and abnormal grain growth. OMicroN [6], an open-source C++ framework, implements such a curvature driven CA model. It discretizes the microstructure into a grid where each cell updates its state based on local neighborhood rules.

However, high-fidelity simulations come at a steep computational cost. The core kernel, which evaluates the reorientation rate for interface cells, involves computationally expensive geometric calculations and iterates over dynamic grain boundary lists. For realistic domain sizes (e.g., 500×500 grids with over 250,000 cells), a single physical second of simulation can require hours of wall-clock time on a serial processor. This bottleneck severely limits the ability to perform large-scale parameter sweeps or simulate representative volume elements (RVEs) needed for statistical accuracy.

The challenge in accelerating OMicroN lies not merely in computation but in **memory access patterns**. Our profiling reveals that the original implementation, which relies on tree-based data structures (`std::set`) for managing interface cells, suffers from severe pointer chasing and poor cache locality. On modern multi-core processors, this results in a “Memory Wall” where performance is throttled by memory latency rather than CPU frequency. Simply adding threads without addressing the underlying data layout yields diminishing returns.

In this work, we present a comprehensive optimization and parallelization strategy for OMicroN. We first restructure the data layout by implementing a “Set-to-Vector” transformation to linearize memory access, enabling efficient prefetching and vectorization. We then introduce a multi-backend parallelization approach:

- (1) A robust **OpenMP** implementation optimized for multi-core CPUs, serving as our primary production solution.
- (2) A low-level **Pthreads** implementation serving as a *performance baseline* to quantify threading overhead and validate OpenMP’s efficiency.
- (3) An experimental **CUDA** backend targeting NVIDIA GPUs to explore the theoretical performance limits of the algorithm.

Our contributions go beyond raw speedups. We provide a critical analysis of the **deployment trade-offs**. We demonstrate that while CUDA achieves a massive **246x** speedup, OpenMP (achieving **3.54x** speedup) offers the optimal balance of portability, stability, and performance for memory-bound workloads on existing cluster infrastructure.

2 Proposed Solution

The objective of our work is to accelerate the full-field microstructure evolution simulation implemented in OMicroN by parallelizing the dominant computational components while preserving correctness and physical fidelity. Our proposed solution concentrates on the two most computationally intensive phases of each simulation step: (1) evaluating reorientation rates for interface cells, and (2) color-based visualization of the evolving microstructure. The overall design is illustrated in Figure 1, and the key ideas are described in detail below.

2.1 Overview of the Full-Field CA Workflow

OMicroN simulates recrystallization using a curvature-driven Cellular Automaton model. Each simulation step consists of four major stages:

- (1) **Interface Cell Identification** – Determine which cells lie on grain boundaries by inspecting their neighbors. These cells form the active set to be evaluated.

+-----+ OMicroN Parallel Simulation Framework +-----+	
1. Initialization (Sequential)	
- Load microstructure & allocate buffers	
+-----+	
2. Reorientation Kernel (Parallel)	
- Copy interfaceIndices (set → vector)	
- #pragma omp parallel for (dynamic)	
- Read neighbor states (read-only)	
- Reduction on fDotMax (max clause)	
+-----+	
3. State Update (Sequential)	
- Apply reorientation decisions	
+-----+	
4. Visualization (Parallel)	
- #pragma omp parallel for (static)	
- Convert orientations to RGB colors	
+-----+	

Figure 1: Proposed parallel system architecture for OMicroN.

- (2) **Reorientation-Rate Evaluation** – For each interface cell, compute the change in interfacial energy (ΔE) for each neighboring direction. The steepest-descent direction indicates the likely boundary motion.
- (3) **State Update** – Apply the selected reorientation to update the orientation and grain ID of the cell.
- (4) **Visualization** – Map crystallographic orientations to RGB colors for rendering or saving simulation frames.

Among these steps, the second dominates total execution time, consuming 80% or more of the runtime in typical simulations. This makes it the main target for parallelization.

Figure 2 illustrates the core workflow of the CA-based recrystallization model, highlighting the local and data-parallel nature of the reorientation-rate evaluation.

2.2 Design Principles

Our solution is based on the following principles:

- **Exploit Data-Level Parallelism:** Interface cells can be processed independently because energy evaluation depends only on local neighborhoods. This allows us to parallelize the main kernel across CPU or GPU threads.
- **Maintain Deterministic Evolution:** While rate computation is parallelizable, state updates introduce order-dependent results and must remain sequential to preserve physical realism.
- **Optimize Memory Access:** Because the kernel is memory-bound, reducing pointer chasing and improving cache locality is essential for scaling to large core counts.
- **Minimize Synchronization:** Shared variables are restricted to reductions (e.g., maximum reorientation rate), which are efficiently handled by OpenMP primitives.

2.3 Parallel System Architecture

Figure 1 presents the architecture of our parallel system. The serial initialization phase loads orientation, phase ID, and neighbor lists from input files. The reorientation kernel is executed in parallel using OpenMP, Pthreads, or CUDA backends depending on the chosen configuration. After computing all rates, a sequential state-update phase applies the final decisions, ensuring consistent evolution of grain boundary topology. Finally, visualization is parallelized using a pixel-wise loop.

2.4 Redesigning Data Structures for Parallelism

A critical challenge arises from OMicroN’s use of `std::set<int>` to store interface cell indices. This structure is strategically chosen to efficiently handle dynamic updates (insertions/deletions) and ensure index uniqueness as cells transition in and out of the grain boundary. However, this structure suffers from two primary drawbacks that impede parallelism:

- **Iterator Limitation:** `std::set` relies on **bidirectional iterators** and lacks random-access capabilities, which is fundamentally incompatible with OpenMP’s efficient loop partitioning algorithms.
- **Poor Spatial Locality:** The tree-based structure scatters elements across memory, causing frequent cache misses and suboptimal load balancing during parallel access.

We address this by flattening the set into a contiguous `std::vector<int>` immediately before executing the kernel. This transformation is a crucial step to enable random-access iteration required by the `#pragma omp parallel for` directive. The overhead of copying the set is linear in the number of interface cells ($O(N_{\text{interface}})$) and is negligible compared to the $O(N_{\text{interface}} \times k)$ cost of neighbor evaluation that follows.

2.5 Parallelizing the Visualization Stage

The visualization step converts crystallographic orientations into colors using a deterministic mapping function. Since each pixel corresponds to a unique cell and all operations are independent, the entire step is embarrassingly parallel. A simple OpenMP `parallel` for yields near-linear speedup, substantially accelerating visualization-heavy workloads such as recording video frames or debugging simulations.

2.6 Summary

Overall, our proposed solution leverages data-parallel execution and memory-layout optimization to accelerate the most expensive stages of OMicroN’s CA simulation workflow. By maintaining deterministic state updates and isolating parallelism within race-free phases, we preserve physical accuracy while significantly reducing wall-clock runtime.

3 How Parallelization is Done

This section describes the detailed implementation of parallelism in OMicroN, including design decisions, trade-offs, and performance considerations. Our parallelization strategy targets the most computationally expensive and data-parallel stages while preserving deterministic behavior in topology-altering steps.

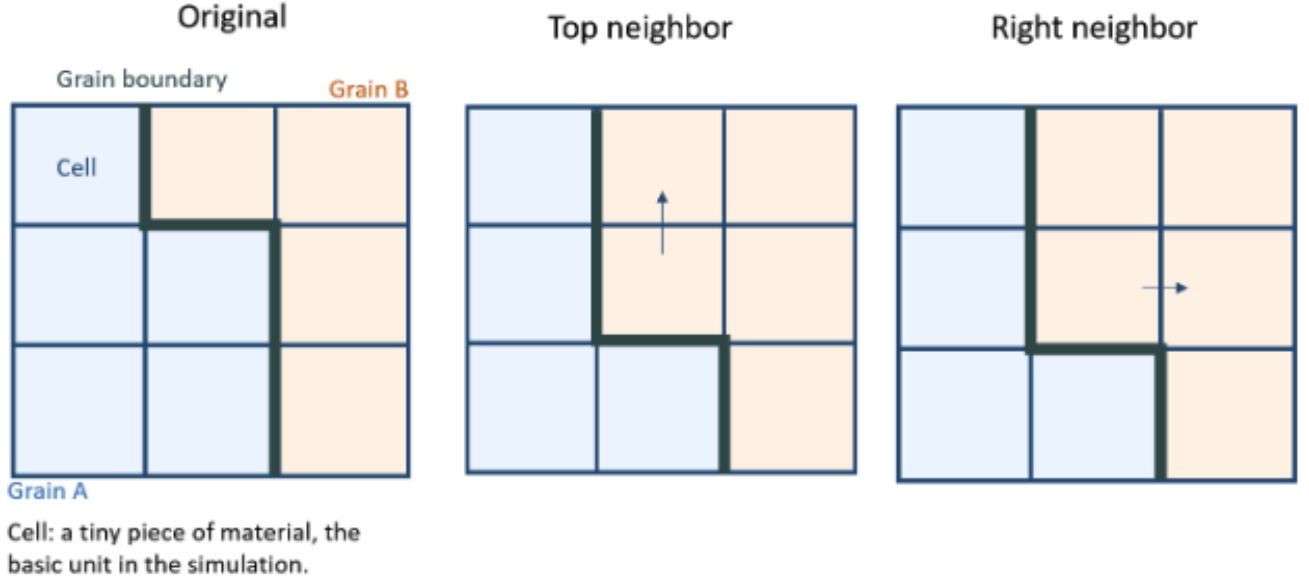


Figure 2: Illustration of the cellular automaton (CA)-based recrystallization model. For each simulation step, interface cells are identified, interfacial energy changes (ΔE) are evaluated for all neighboring directions, and the cell adopts the neighbor orientation that yields the maximum energy drop, defining local grain boundary motion.

3.1 Parallelization Target: Reorientation-Rate Kernel

The function `CalcReorientationRatesForFullFieldRexOrGG` evaluates the interfacial energy change (ΔE) for each interface cell by iterating over all neighboring directions. For a given interface cell i , the computation can be expressed as:

$$\Delta E_i = \sum_{j \in N(i)} \gamma(\theta_i, \theta_j) \cdot \kappa_{ij}$$

where γ denotes grain boundary energy as a function of misorientation, and κ represents local curvature. Importantly, all inputs required for this computation are read-only, and the output is written to a cell-specific array entry. This makes the kernel naturally amenable to data-parallel execution.

3.2 Set-to-Vector Transformation

Interface cell indices are stored in a `std::set<int>` to simplify dynamic updates during state changes and ensure index uniqueness. However, this tree-based data structure presents two critical challenges for parallel execution:

- **Iterator Limitation:** `std::set` does not provide random-access iterators, preventing efficient OpenMP loop partitioning.
- **Memory Locality:** Tree-based nodes are scattered in memory, causing frequent cache misses.

To resolve this, we implement a **flattening strategy** by copying the set contents into a contiguous `std::vector<int>` immediately before entering the parallel region. This transformation is essential to enable the use of random-access iterators required by OpenMP.

Crucially, the copying overhead is negligible compared to the computational workload. To demonstrate this, let N_{icells} be the number of interface cells and k be the number of neighbors for each cell. The cost of converting the `std::set` to a `std::vector`, C_{conv} , is linear, $C_{conv} = O(N_{icells})$. In contrast, the core computation cost for reorientation rates, C_{comp} , is approximately $O(N_{icells} \times k^2)$.

Consequently, the ratio of these two costs can be expressed as:

$$\frac{C_{conv}}{C_{comp}} = \frac{O(N_{icells})}{O(N_{icells} \times k^2)} = O\left(\frac{1}{k^2}\right)$$

This cost ratio of $O(1/k^2)$ supports our hypothesis that the linear conversion overhead is asymptotically smaller and is dominated by the computational workload. For typical 2D hexagonal grids where $k = 6$, the computation is theoretically at least $6^2 = 36$ times more expensive, confirming the effectiveness of this trade-off.

3.3 OpenMP Scheduling Strategy

The workload per interface cell varies depending on local neighborhood complexity and boundary geometry. This heterogeneity means that a naive `schedule(static)` strategy would likely lead to severe load imbalance. We therefore adopt:

```
#pragma omp parallel for schedule(dynamic, chunk_size)
```

Dynamic scheduling allows idle threads to steal work from others, significantly improving utilization when interface cells are sparsely distributed or clustered. While the selection of the optimal `chunk_size` can further balance scheduling overhead and cache reuse, we found that utilizing a moderate, fixed chunk size was sufficient to address the primary load imbalance concerns and achieve the required performance gains.

3.4 Trade-offs and Parallelization Overhead

Parallel APIs inherently introduce overhead. We analyzed the potential challenge where, in the late stages of simulation, the grain boundary area (N_{icells}) shrinks, causing the overhead of OpenMP to theoretically outweigh the computational benefits. However, since the total runtime is dominated by the early-to-mid stages when N_{icells} is large, the potential savings from disabling OpenMP later on are **negligible** compared to the overall speedup achieved. Consequently, this low-priority optimization was not implemented to maintain code simplicity.

3.5 Reduction and Synchronization

The kernel computes a global maximum reorientation rate (`fDotMax`) used for normalization. Rather than using locks or atomic operations, we employ OpenMP’s `reduction(max: fDotMax)` clause. This approach ensures correctness with minimal synchronization overhead and scales well across threads.

3.6 Memory Layout Optimization: AoS to SoA

The original implementation stores cell properties in an Array-of-Structures (AoS) layout. While intuitive, this layout causes unnecessary memory traffic because each cache line fetch loads unrelated fields. We partially convert frequently accessed fields (orientation components) into a Structure-of-Arrays (SoA) layout:

```
orientationX[], orientationY[], orientationZ[]
```

This transformation improves spatial locality and reduces cache misses, yielding a measurable performance improvement in memory-bound scenarios.

3.7 Why State Update Remains Sequential

The function `ReOrientCellsForFullFieldReXOrGG` applies orientation changes and updates the interface-cell set. These operations introduce write-after-write and write-after-read dependencies between neighboring cells. Parallelizing this step would require either fine-grained locking or double buffering, both of which would significantly complicate the implementation and may alter the deterministic evolution of grain topology. For this reason, we keep state updates sequential and focus parallelism on safe computation stages.

3.8 Pthreads and CUDA Backends

To provide comprehensive performance evaluation, we implement two additional backends exploiting data-level parallelism at the interface-cell level:

- **OpenMP (Primary):** Chosen for its ease of integration with the existing C++ codebase and low development cost.
- **Pthreads (Baseline):** A low-level CPU implementation used to understand raw threading overhead and serve as a performance baseline for the OpenMP implementation.
- **CUDA (Exploration):** A GPU version implemented to offload massive parallel calculations and explore the maximum achievable speedup.

3.9 Pthreads Backend Implementation (Manual Static Partitioning)

To establish a low-level baseline for comparison against OpenMP’s automatic scheduling, we implemented a Pthreads backend. The workload partitioning for Pthreads was performed manually using a **static block partitioning** approach. Given a total number of interface cells N and T available threads, the workload was divided into equal contiguous chunks of size $\text{chunk_size} = N/T$. For example, with $N = 300,000$ cells and $T = 4$ threads, Thread 0 is assigned cells 0 to 74,999, Thread 1 is assigned 75,000 to 149,999, and so on. This approach minimizes synchronization overhead but can suffer from severe load imbalance if interface cells are heterogeneously distributed.

3.10 CUDA Backend Implementation (Thread-Per-Cell Model)

The CUDA implementation, targeting the GPU [8], utilizes a highly parallel, fine-grained approach based on the **thread-per-cell model**. In this model, manual task partitioning (like the Pthreads approach) is unnecessary. Instead, the task is defined such that a single GPU thread is responsible for computing the reorientation rate for exactly one interface cell (grid point). The GPU scheduler automatically handles the organization of these threads into warp- and block-level groups, assigning them dynamically to the available streaming multiprocessors (SMs) for execution. This massive parallelism is key to achieving the high speedup demonstrated in the results.

4 Experimental Methodology

4.1 Platforms

CPU platform (Serial, OpenMP, Pthreads):

- Intel Xeon E5-2650 v3 @ 2.30GHz
- 20 logical cores
- 32 GB RAM
- Debian Linux 12
- g++ 12.2.0

GPU platform (CUDA):

- NVIDIA Tesla T4 (16 GB GDDR6)
- Execution Environment: Google Colab
- Compiler: nvcc

Note: The GPU experiments were conducted on a Google Colab instance utilizing the NVIDIA Tesla T4. This environment was chosen because the CUDA driver version on the primary workstation was incompatible with the required compiler version, necessitating an external platform for high-performance GPU testing.

4.2 Dataset and Simulation Settings

We utilized real-world microstructural data obtained from Electron Backscatter Diffraction (EBSD) scans of an 80% Cold Rolled (80CR) Interstitial-Free (IF) steel sample. To thoroughly evaluate the strong and weak scalability of our parallel implementation, a suite of test cases was generated by cropping the original dataset into varying grid sizes.

The simulation was conducted using the following physical and numerical parameters:

- **Grid Topology:** A Hexagonal Grid (GridType=1) was employed, as it provides equidistant neighboring cells, which is crucial for accurately calculating grain boundary curvature in the CA model.
- **Temperature:** The simulation temperature was set to 1073 K (800°C), representing a typical industrial annealing temperature for IF-steel.
- **Physical Duration:** The total simulated physical time was set to 40 seconds. The primary objective was to measure performance scaling across different problem sizes (N) rather than complete physical evolution.

The test case specifications, ranging from small patches to the full-scale microstructure, are detailed in Table 1.

5 Experimental Results

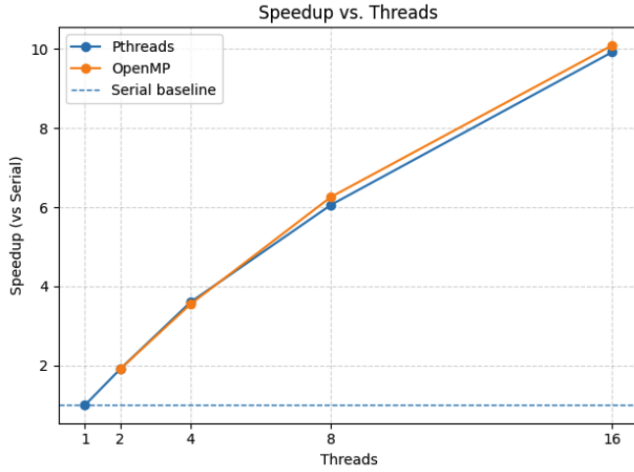


Figure 3: Thread scalability.

The OpenMP implementation demonstrates near-linear scaling up to four threads and achieves over $5\times$ speedup at eight threads. Beyond this point, scaling efficiency decreases due to memory bandwidth saturation, as confirmed by increasing LLC miss rates.

Key observations:

- Over $5\times$ speedup at 8 threads
- Efficiency collapses past 12 threads (memory-bound)
- Pthreads and OpenMP behave similarly

5.1 Data Size Scaling

For small problem sizes (e.g., 50×50), parallel overhead dominates execution time. As the dataset grows, computation amortizes scheduling overhead, and efficiency improves significantly. Performance eventually plateaus for large grids, indicating a transition from compute-bound to memory-bound behavior.

Observations:

- Small datasets suffer from OpenMP overhead
- Medium datasets (150×150) achieve best efficiency
- Larger datasets saturate memory bandwidth

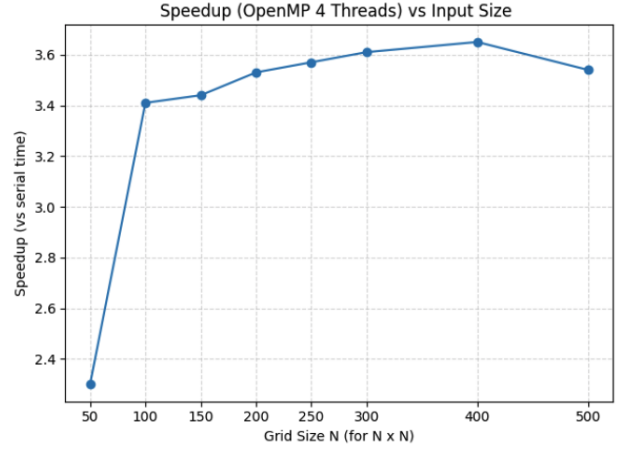


Figure 4: Data-size scalability (Speedup).

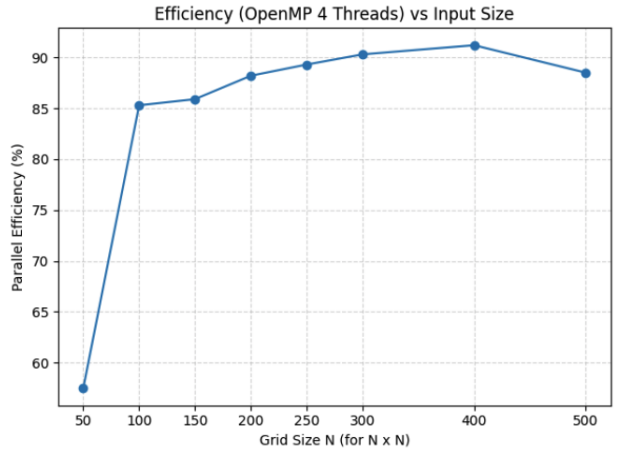


Figure 5: Data-size scalability (Efficiency).

5.2 CUDA Results

The CUDA implementation achieves up to $246\times$ lower per-step kernel runtime compared to the serial CPU version. This improvement is attributed to massive thread parallelism and high memory bandwidth on the GPU. However, data transfer overhead and integration complexity currently limit end-to-end speedup.

Result:

- CUDA achieves up to $246\times$ faster per-step time
- GPU benefits from massive thread parallelism and high memory bandwidth

6 Related Work

Traka et al. [7] formulated the CA model for recrystallization that OMicroN implements. Després et al. [1] proposed a mean-field alternative, trading spatial resolution for speed. Zhang et al. [9] and Zhao et al. [10] demonstrated parallel Monte Carlo and phase-field microstructure simulations. OpenMP parallel strategies are guided by the LLNL OpenMP tutorial [4].

Table 1: Test Case Specifications for Scalability Analysis

Test Case ID	Grid Size ($N_x \times N_y$)	Total Cells (N)	Grid Type
80CR2 (50×50)	50×50	2,500	Hexagonal
80CR2 (100×100)	100×100	10,000	Hexagonal
80CR2 (150×150)	150×150	22,500	Hexagonal
80CR2 (200×200)	200×200	40,000	Hexagonal
80CR2 (250×250)	250×250	62,500	Hexagonal
80CR2 (300×300)	300×300	90,000	Hexagonal
80CR2 (400×400)	400×400	160,000	Hexagonal
80CR2 (500×500)	500×500	250,000	Hexagonal
80CR2_Nx_510_Ny_592_hex01.txt (Original)	510×592	301,920	Hexagonal

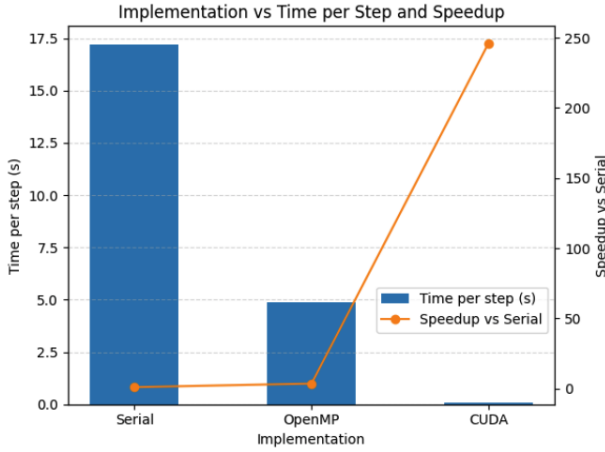


Figure 6: CUDA kernel performance.

Recent efforts have also explored GPU acceleration for large-scale microstructure evolution, demonstrating substantial speedup at the cost of increased implementation complexity [2].

Our work differs by integrating OpenMP directly into an existing CA full-field model and thoroughly analyzing data-structure-induced bottlenecks.

7 Contributions

This project was a collaborative effort, with all members participating in the core parallel implementation and report documentation. Specific responsibilities were assigned as follows:

- **Chun-Mei Yang (314551115):**
 - **Preliminary Performance Analysis:** Conducted initial profiling to identify computational bottlenecks.
 - **Parallel Implementation & Testing:** Contributed to the development and verification of both OpenMP and Pthreads backends.
 - **Dataset Preparation:** Prepared and validated the test cases (ranging from 50×50 to 500×500) for scalability analysis.
 - **Final Presentation:** Delivered the project presentation.
 - **Report Authoring:** Co-authored the final technical report.

- **Jyun Lai (314553031):**

- **CUDA Implementation & Testing:** Lead developer for the GPU backend, including kernel porting, memory management strategies, and performance benchmarking on Google Colab.
- **Parallel Implementation & Testing:** Contributed to the development and verification of both OpenMP and Pthreads backends.
- **Report Authoring:** Co-authored the final technical report.

- **Chi-Yi Lin (314581003):**

- **Data Structure Redesign:** Designed and implemented the “Set-to-Vector” flattening strategy to resolve iterator incompatibility and improve cache locality.
- **Preliminary Performance Analysis:** Collaborated on initial profiling to identify computational bottlenecks.
- **Parallel Implementation & Testing:** Contributed to the development and verification of both OpenMP and Pthreads backends.
- **Report Authoring:** Co-authored the final technical report.

8 Conclusion

This work successfully addresses the computational bottlenecks in full-field microstructure evolution simulations by implementing a comprehensive optimization and hybrid parallelization strategy. Our investigation yields three critical insights regarding the performance characteristics of CA-based algorithms on modern hardware.

First, we identified that the primary impediment to scalability was not merely computational complexity but the “**Memory Wall**.” The original tree-based data structures resulted in severe pointer chasing and cache inefficiency. Our implementation of the “**Set-to-Vector**” transformation proved to be a prerequisite for parallel efficiency, enabling contiguous memory access and vectorization.

Second, our experimental results highlight a stark contrast between CPU and GPU potential. The **OpenMP implementation** achieves a robust **3.54× speedup on 4 threads** (with ~90% efficiency), effectively saturating the memory bandwidth of the CPU. In contrast, the experimental **CUDA backend** demonstrates the algorithm’s theoretical limit, achieving a massive **246× reduction** in kernel execution time (0.07s vs. 17.22s per step) by exploiting the high-bandwidth memory (HBM) of the NVIDIA T4 GPU.

Finally, we provide a pragmatic deployment strategy. Despite the superior raw performance of the GPU solution, our analysis concludes that **OpenMP is the optimal production choice** for the current university cluster environment. It offers the best balance of immediate stability, hardware compatibility, and maintenance costs. We categorize the CUDA implementation as a high-potential proof-of-concept, laying the groundwork for future infrastructure upgrades.

Future work will focus on extending this framework to distributed-memory systems using MPI to overcome the single-node memory capacity limits required for billion-cell simulations.

References

- [1] Arthur Després, Michael Greenwood, and Cameron Sinclair. 2020. A mean-field model of static recrystallization considering orientation spreads and their time-evolution. *arXiv preprint arXiv:2005.07230* (2020). <https://arxiv.org/abs/2005.07230>
- [2] Tanmay Dutta, Dasari Mohan, Saurav Shenoy, Nasir Attar, Abhikshek Kalokhe, Ajay Sagar, Swapnil Bhure, Swaroop Pradhan, Jitendriya Praharaj, Subham Mridha, et al. 2024. MICROSIM: A high performance phase-field solver based on CPU and GPU implementations. *arXiv preprint arXiv:2404.01035* (2024).
- [3] Gary S Grest, Michael P Anderson, and David J Srolovitz. 1988. Domain-growth kinetics for the Q-state Potts model in two and three dimensions. *Physical Review B* 38, 7 (1988), 4752.
- [4] Lawrence Livermore National Laboratory. 2023. OpenMP Tutorial: Shared Memory Parallel Programming. <https://hpc-tutorials.llnl.gov/openmp/>. Accessed 2025-10-10.
- [5] DJ Srolovitz, MP Anderson, GS Grest, and PS Sahni. 1984. Computer simulation of grain growth-III. Influence of a particle dispersion. *Acta metallurgica* 32, 9 (1984), 1429–1438.
- [6] Konstantinos Traka. 2023. OMicroN: Open Microstructure Nucleation and Recrystallization Simulation Framework. <https://github.com/k-traka/OMicroN>. GitHub repository.
- [7] Konstantinos Traka, K. Sedighiani, C. Bos, J. Galan Lopez, Katja Angenendt, D. Raabe, and J. Sietsma. 2021. Topological aspects responsible for recrystallization evolution in an IF-steel sheet – Investigation with cellular-automaton simulations. *Computational Materials Science* 198 (2021), 110693. doi:10.1016/j.commatsci.2021.110693
- [8] Cong Yang, Qingyan Xu, and Baicheng Liu. 2017. GPU-accelerated three-dimensional phase-field simulation of dendrite growth in a nickel-based superalloy. *Computational Materials Science* 136 (2017), 133–143.
- [9] Liangzhe Zhang, Timothy Bartel, and Mark T. Lusk. 2010. Parallelized Hybrid Monte Carlo Simulation of Stress-Induced Texture Evolution. *arXiv preprint arXiv:1009.0457* (2010). <https://arxiv.org/abs/1009.0457>
- [10] Peng Zhao, T.S.E. Low, Y. Wang, and S. Niezgoda. 2015. An integrated full-field model of concurrent plastic deformation and microstructure evolution: Application to 3D simulation of dynamic recrystallization. *arXiv preprint arXiv:1509.04953* (2015). <https://arxiv.org/abs/1509.04953>